

The Scope and Lifetime of Variables, operators, Shorthand Assignments

Types of Variable

- There are three types of variables in java:
 - local variable
 - instance variable
 - static variable

Local Variable

- A variable which is declared inside the method is called local variable.

instance variable

- A variable which is declared inside the class but outside the method, is called instance variable . It is not declared as static.

static variable

- A variable that is declared as static is called static variable. It cannot be local.

- Example to understand the types of variables in java

```
class A{  
    int data=50;//instance variable  
    static int m=100;//static variable  
    void method(){  
        int n=90;//local variable  
    }  
} //end of class
```

Scope of Variables

- **Scope** refers to *where a variable can be accessed* in a program.
- The scope of a variable defines the section of the code in which the variable is visible. As a general rule, variables that are defined within a block are not accessible outside that block
 - 1. Local Scope
 - 2. Block Scope
 - 3. Global Scope
 - 4. Function Parameters

Local Scope

- A variable declared inside a function or block {}.

```
void myFunction() {  
    int x = 10; // Local variable  
}
```

- **Accessible only inside myFunction**
- **Cannot be used outside the function**

Block Scope

- A variable declared inside a block like if, for, etc.

```
if (true) {  
    int y = 5;  
}
```

- **y exists only inside the if block**

Global Scope

- A variable declared outside all functions.

```
int globalVar = 100;
```

```
void myFunction() {  
    printf("%d", globalVar);  
}
```

- **Accessible from anywhere in the file**

Function Parameters

- Parameters are local to the function.

```
void add(int a, int b) {  
    int sum = a + b;  
}
```

- **a, b, and sum exist only inside add**

Lifetime of Variables

- **Lifetime** refers to *how long a variable exists in memory* during program execution.
- The lifetime of a variable refers to how long the variable exists before it is destroyed.
- Destroying variables refers to de-allocating the memory that was allotted to the variables when declaring it.
 - **Automatic Variables**
 - Static Variables
 - Global Variables

Automatic Variables

- Created when function starts
- Destroyed when function ends

```
void test() {  
    int x = 5; // Created here  
}           // Destroyed here
```

Static Variables

- Retain their value between function calls
- Exist for entire program duration

```
void counter() {  
    static int count = 0;  
    count++;  
    printf("%d\n", count);  
}
```

- If called multiple times:

1
2
3

Operators in java

- Operator in java is a symbol that is used to perform operations. For example: +, -, *, / etc.
- There are many types of operators in java which are given below:
 - Unary Operator,
 - Arithmetic Operator,
 - shift Operator,
 - Relational Operator,
 - Bitwise Operator,
 - Logical Operator,
 - Ternary Operator and
 - Assignment Operator.

Operators Hierarchy

Operator Precedence

Operators	Precedence
postfix	<code>expr++ expr--</code>
unary	<code>++expr --expr +expr -expr ~ !</code>
multiplicative	<code>* / %</code>
additive	<code>+ -</code>
shift	<code><< >> >>></code>
relational	<code>< > <= >= instanceof</code>
equality	<code>== !=</code>
bitwise AND	<code>&</code>
bitwise exclusive OR	<code>^</code>
bitwise inclusive OR	<code> </code>
logical AND	<code>&&</code>
logical OR	<code> </code>
ternary	<code>? :</code>
assignment	<code>= += -= *= /= %= &= ^= = <<= >>= >>>=</code>

Expressions

- Expressions are essential building blocks of any Java program, usually created to produce a new value, although sometimes an expression simply assigns a value to a variable. Expressions are built using values, variables, operators and method calls.

Types of Expressions

- While an expression frequently produces a result, it doesn't always. There are three types of expressions in Java:
 - Those that produce a value, i.e. the result of $(1 + 1)$
 - Those that assign a variable, for example $(v = 10)$
 - Those that have no result but might have a "side effect" because an expression can include a wide range of elements such as method invocations or increment operators that modify the state (i.e. memory) of a program.

Java Type casting and Type conversion

- Widening or Automatic Type Conversion (implicit)
 - Widening conversion takes place when two data types are automatically converted. This happens when:
 - The two data types are compatible.
 - When we assign value of a smaller data type to a bigger data type.
 - For Example, in java the numeric data types are compatible with each other but no automatic conversion is supported from numeric type to char or boolean. Also, char and boolean are not compatible with each other.

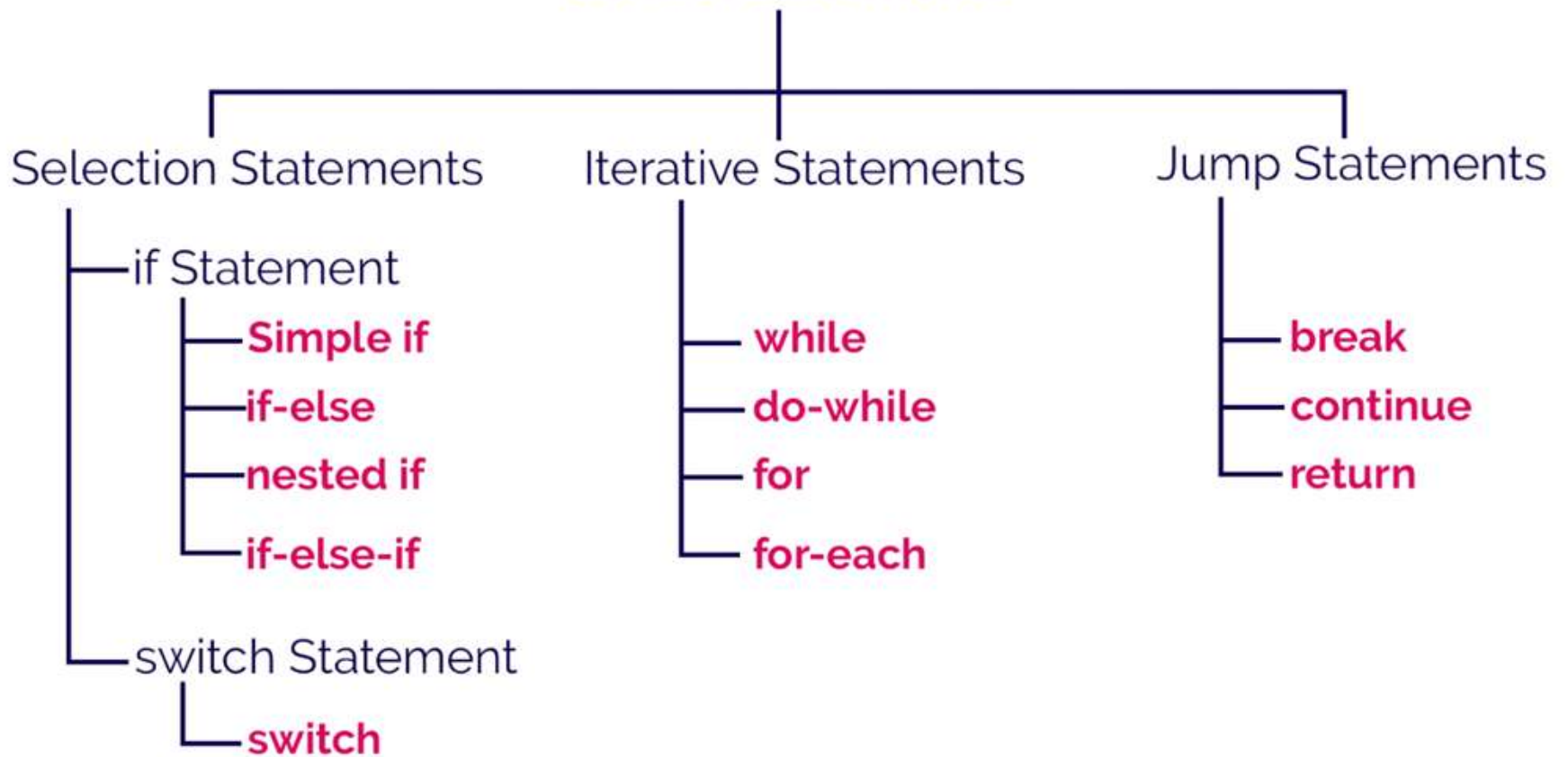
Narrowing or Explicit Conversion

- If we want to assign a value of larger data type to a smaller data type we perform explicit type casting or narrowing.
- This is useful for incompatible data types where automatic conversion cannot be done. [?] Here, target-type specifies the desired type to convert the specified value to.

Control Statements

- In java, the default execution flow of a program is a sequential order.
- It may not be suitable for all situations.
- Sometimes, we may want to jump from line to another line, we may want to skip a part of the program, or sometimes we may want to execute a part of the program again and again.
- To solve this problem, java provides control statements.

Control Statements



Selection Statement

- Selection statements use conditions that are Boolean expressions.
 - A Boolean expression is an expression that evaluates to a Boolean value: true or false.

boolean Data Type

- The boolean data type declares a variable with the value either **true** or **false**.
- How do you compare two values, such as whether a radius is greater than 0, equal to 0, or less than 0?
- Java provides six relational operators (also known as comparison operators)

Relational Operators

<i>Java Operator</i>	<i>Mathematics Symbol</i>	<i>Name</i>	<i>Example (radius is 5)</i>	<i>Result</i>
<	<	Less than	<code>radius < 0</code>	<code>false</code>
<=	≤	Less than or equal to	<code>radius <= 0</code>	<code>false</code>
>	>	Greater than	<code>radius > 0</code>	<code>true</code>
>=	≥	Greater than or equal to	<code>radius >= 0</code>	<code>true</code>
==	=	Equal to	<code>radius == 0</code>	<code>false</code>
!=	≠	Not equal to	<code>radius != 0</code>	<code>true</code>

if Statement

- In java, we use the if statement to test a condition and decide the execution of a block of statements based on that condition result.
- Java has several types of selection statements:
 - one-way if statements,
 - two way if-else statements,
 - nested if statements,
 - multi-way if-else statements,
 - switch statements, and
 - conditional operators.

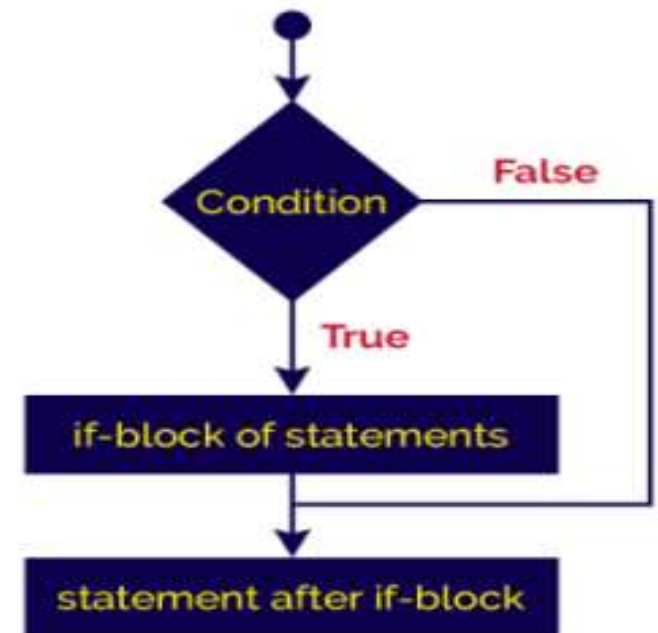
- The **if** statement checks, the given condition then decides the execution of a block of statements.
- If the condition is True, then the block of statements is executed and if it is False, then the block of statements is ignored.

- The syntax and execution flow of if the statement is as follows.

Syntax

```
if(condition){  
    if-block of statements;  
    ...  
}  
statement after if-block;  
...
```

Flow of execution



```
import java.util.Scanner;
```

```
public class IfStatementTest {
```

```
    public static void main(String[] args) {
```

```
        Scanner read = new Scanner(System.in);
```

```
        System.out.print("Enter any number: ");
```

```
        int num = read.nextInt();
```

```
        if((num % 5) == 0) {
```

```
            System.out.println("We are inside the if-block!");
```

```
            System.out.println("Given number is divisible by 5!!");
```

```
        }
```

```
        System.out.println("We are outside the if-block!!!");
```

```
    }
```

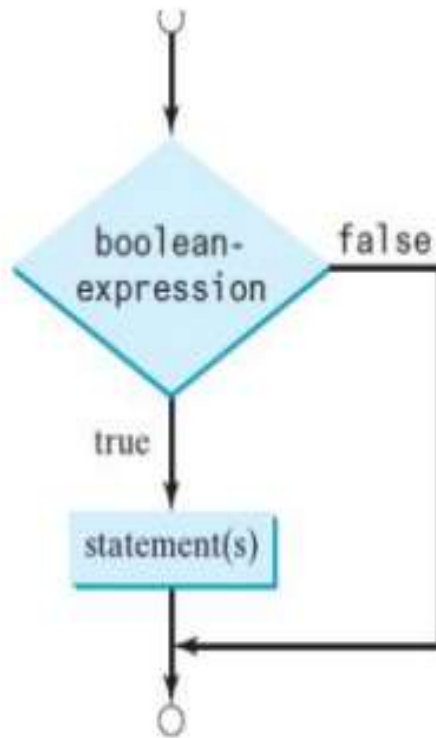
```
}
```

In this execution, the number 12 is not divisible by 5. So, the condition becomes False and the condition is evaluated as False. Then the if statement ignores the execution of its block of statements.

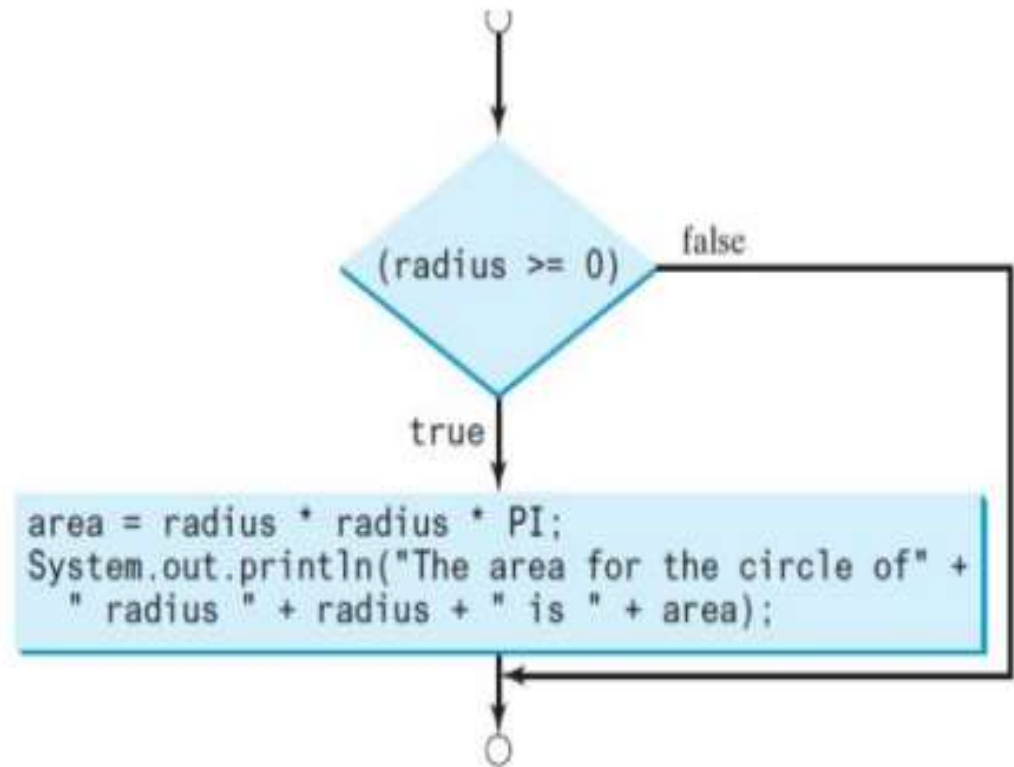
one-way if statements

- A one-way if statement executes an action if and only if the condition is true. The syntax for a one-way if statement is as follows:

```
if (boolean-expression) {  
    statement(s);  
}
```



(a)



(b)

The boolean-expression is enclosed in parentheses.

If the boolean-expression evaluates to true, the statements in the block are executed.

```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area for  
the circle of radius " +  
radius + " is " + area);  
}
```

- The boolean-expression is enclosed in parentheses.
- For example, the code in (a) is wrong. It should be corrected. as shown in (b).

```
if i > 0 {  
    System.out.println("i is positive");  
}
```

(a) Wrong

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(b) Correct

- The block braces can be omitted if they enclose a single statement. For example, the following statements are equivalent:

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(a)

Equivalent

```
if (i > 0)  
    System.out.println("i is positive");
```

(b)

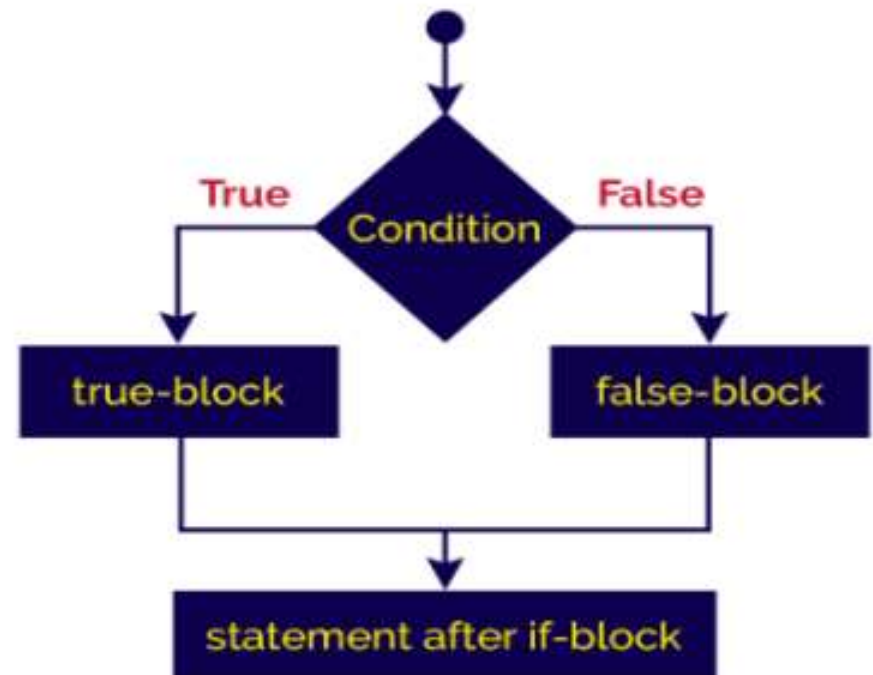
Two-Way if-else Statements

- An if-else statement decides the execution path based on whether the condition is true or false.
- A one-way if statement performs an action if the specified condition is true. If the condition is false, nothing is done.
- But what if you want to take alternative actions when the condition is false? You can use a two-way if-else

Syntax

```
if(condition){  
    true-block of statements;  
    ...  
}  
else{  
    false-block of statements;  
    ...  
}  
statement after if-block;  
...
```

Flow of execution



```
import java.util.Scanner;

public class IfElseStatementTest {

    public static void main(String[] args) {

        Scanner read = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int num = read.nextInt();

        if((num % 2) == 0) {
            System.out.println("We are inside the true-block!");
            System.out.println("Given number is EVEN number!!");
        }
        else {
            System.out.println("We are inside the false-block!");
            System.out.println("Given number is ODD number!!");
        }

        System.out.println("We are outside the if-block!!!");

    }

}
```

When we run this code, it produces the following output.

A screenshot of a Java console window titled 'Console'. The window shows the execution of a Java application named 'IfElseStatementTest'. The output text is as follows:

```
<terminated> IfElseStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\ja  
Enter any number: 16  
We are inside the true-block!  
Given number is EVEN number!!  
We are outside the if-block!!!
```

The number '16' is highlighted in green in the original image. The console window has a standard Windows-style title bar and a toolbar with various icons. A 'Quick Access' button is visible in the top right corner of the window.

Nested if and Multi-Way if-else Statements

- An if statement can be inside another if statement to form a nested if statement.
- The statement in an if or if-else statement can be any legal Java statement, including another if or if-else statement.
- The inner if statement is said to be nested inside the outer if statement.
- The inner if statement can contain another if statement; in fact, there is no limit to the depth of the nesting.

For example, the following is a nested if statement:

```
if (i > k) {  
    if (j > k)  
        System.out.println("i and j are greater than k");  
}  
else  
    System.out.println("i is less than or equal to k");
```

The if (j > k) statement is nested inside the if (i > k) statement.

The nested if statement can be used to implement multiple alternatives.

Multiway if-else statement

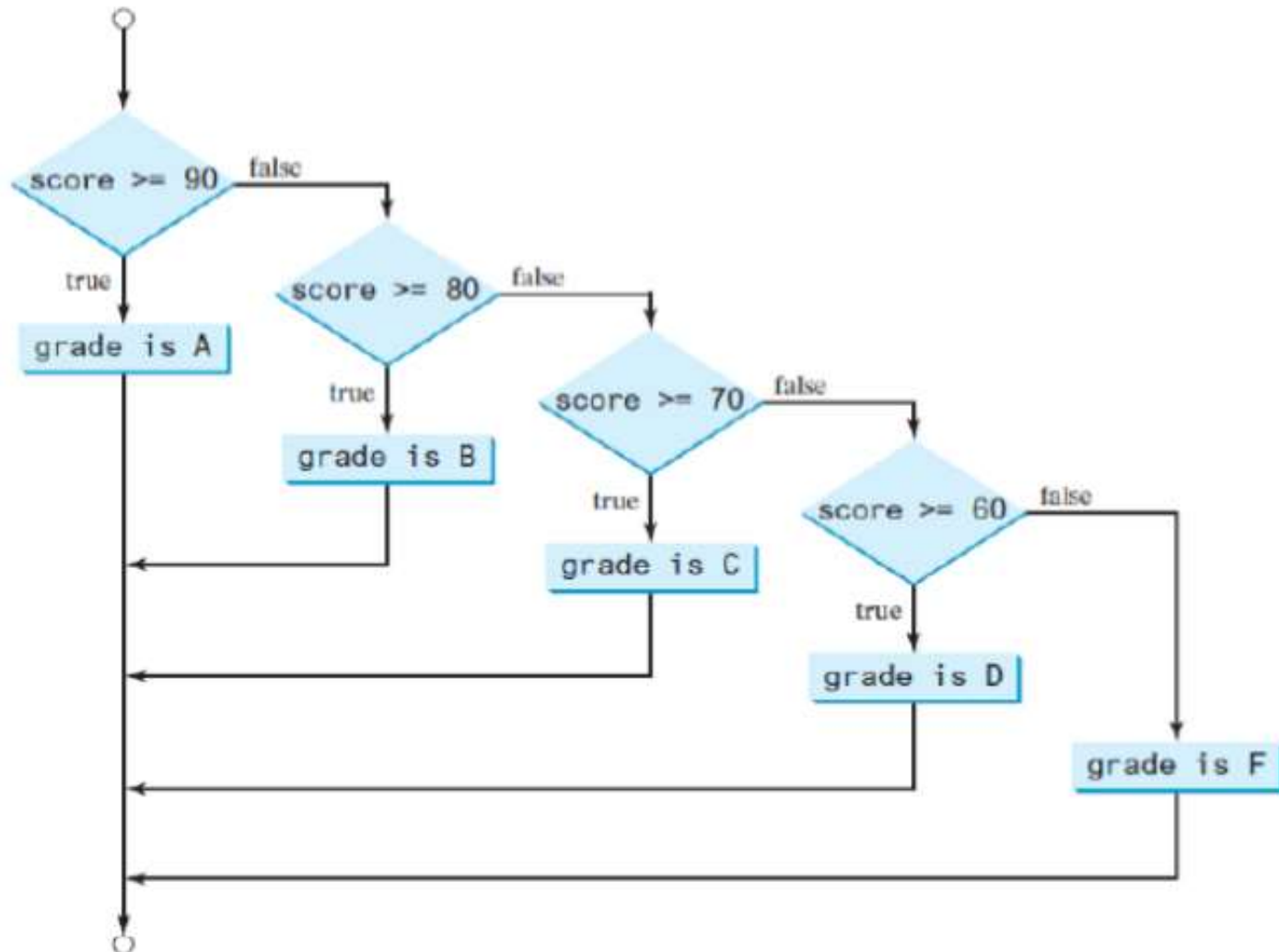


FIGURE 3.4 You can use a multi-way **if-else** statement to assign a grade.

```
if (score >= 90)
    System.out.print("A");
else
    if (score >= 80)
        System.out.print("B");
    else
        if (score >= 70)
            System.out.print("C");
        else
            if (score >= 60)
                System.out.print("D");
            else
                System.out.print("F");
```

(a)

Equivalent

This is better

```
if (score >= 90)
    System.out.print("A");
else if (score >= 80)
    System.out.print("B");
else if (score >= 70)
    System.out.print("C");
else if (score >= 60)
    System.out.print("D");
else
    System.out.print("F");
```

(b)

FIGURE 3.3 A preferred format for multiple alternatives is shown in (b) using a multi-way `if-else` statement.

Logical Operators

- Sometimes, whether a statement is executed is determined by a combination of several conditions.
- You can use logical operators to combine these conditions to form a compound Boolean expression.
- Logical operators, also known as Boolean operators, operate on Boolean values to create a new Boolean value

A program that checks whether a number is divisible by 2 and 3, by 2 or 3, and by 2 or 3 but not both.

```
1  import java.util.Scanner;
2
3  public class TestBooleanOperators {
4      public static void main(String[] args) {
5          // Create a Scanner
6          Scanner input = new Scanner(System.in);
7
8          // Receive an input
9          System.out.print("Enter an integer: ");
10         int number = input.nextInt();
11
12         if (number % 2 == 0 && number % 3 == 0)
13             System.out.println(number + " is divisible by 2 and 3.");
14
15         if (number % 2 == 0 || number % 3 == 0)
16             System.out.println(number + " is divisible by 2 or 3.");
17
18         if (number % 2 == 0 ^ number % 3 == 0)
19             System.out.println(number +
20                 " is divisible by 2 or 3, but not both.");
21     }
22 }
```

```
Enter an integer: 4   
4 is divisible by 2 or 3.  
4 is divisible by 2 or 3, but not both.
```

```
Enter an integer: 18   
18 is divisible by 2 and 3.  
18 is divisible by 2 or 3.
```

$(\text{number} \% 2 == 0 \ \&\& \ \text{number} \% 3 == 0)$ (line 12) checks whether the number is divisible by both 2 and 3.

$(\text{number} \% 2 == 0 \ || \ \text{number} \% 3 == 0)$ (line 15) checks whether the number is divisible by 2 or by 3.

$(\text{number} \% 2 == 0 \ \wedge \ \text{number} \% 3 == 0)$ (line 18) checks whether the number is divisible by 2 or 3, but not both.

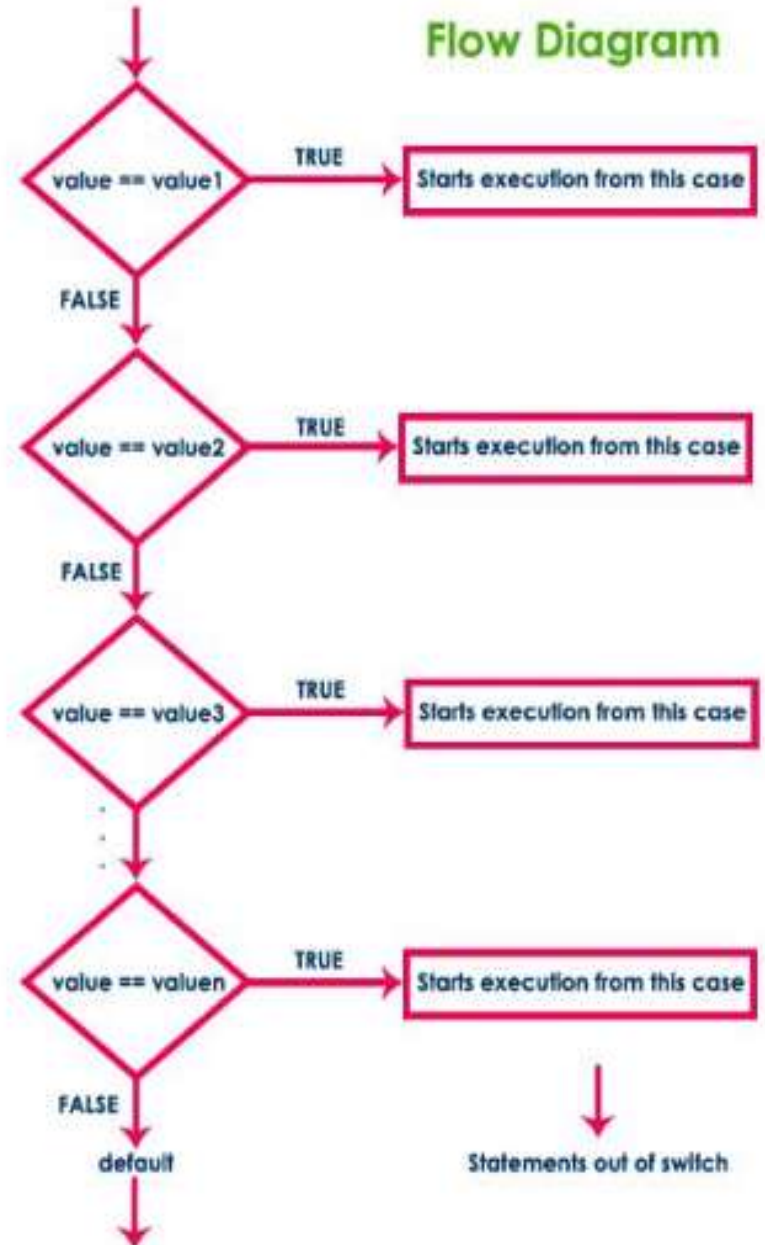
switch Statements

- Using the switch statement, one can select only one option from more number of options very easily.
- In the switch statement, we provide a value that is to be compared with a value associated with each option.
- Whenever the given value matches the value associated with an option, the execution starts from that option.
- In the switch statement, every option is defined as a case.

Syntax

```
switch ( expression or value )  
{  
    case value1: set of statements;  
        ....  
    case value2: set of statements;  
        ....  
    case value3: set of statements;  
        ....  
    case value4: set of statements;  
        ....  
    case value5: set of statements;  
        ....  
    .  
    .  
    default: set of statements;  
}
```

Flow Diagram





SwitchStatementTest.java

```
1 import java.util.Scanner;
2
3 public class SwitchStatementTest {
4
5     public static void main(String[] args) {
6
7         Scanner read = new Scanner(System.in);
8         System.out.print("Press any digit: ");
9
10        int value = read.nextInt();
11
12        switch( value )
13        {
14            case 0: System.out.println("ZERO") ; break ;
15            case 1: System.out.println("ONE") ; break ;
16            case 2: System.out.println("TWO") ; break ;
17            case 3: System.out.println("THREE") ; break ;
18            case 4: System.out.println("FOUR") ; break ;
19            case 5: System.out.println("FIVE") ; break ;
20            case 6: System.out.println("SIX") ; break ;
21            case 7: System.out.println("SEVEN") ; break ;
22            case 8: System.out.println("EIGHT") ; break ;
23            case 9: System.out.println("NINE") ; break ;
24            default: System.out.println("Not a Digit") ;
25        }
26
27    }
28
29 }
30
```

Console

<terminated> SwitchStatementTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\java.exe
Press any digit: 8
EIGHT

iterative statements

- The java programming language provides a set of iterative statements that are used to execute a statement or a block of statements repeatedly as long as the given condition is true.
- The iterative statements are also known as looping statements or repetitive statements.
- Java provides the following iterative statements.
 - while statement
 - do-while statement
 - for statement
 - for-each statement

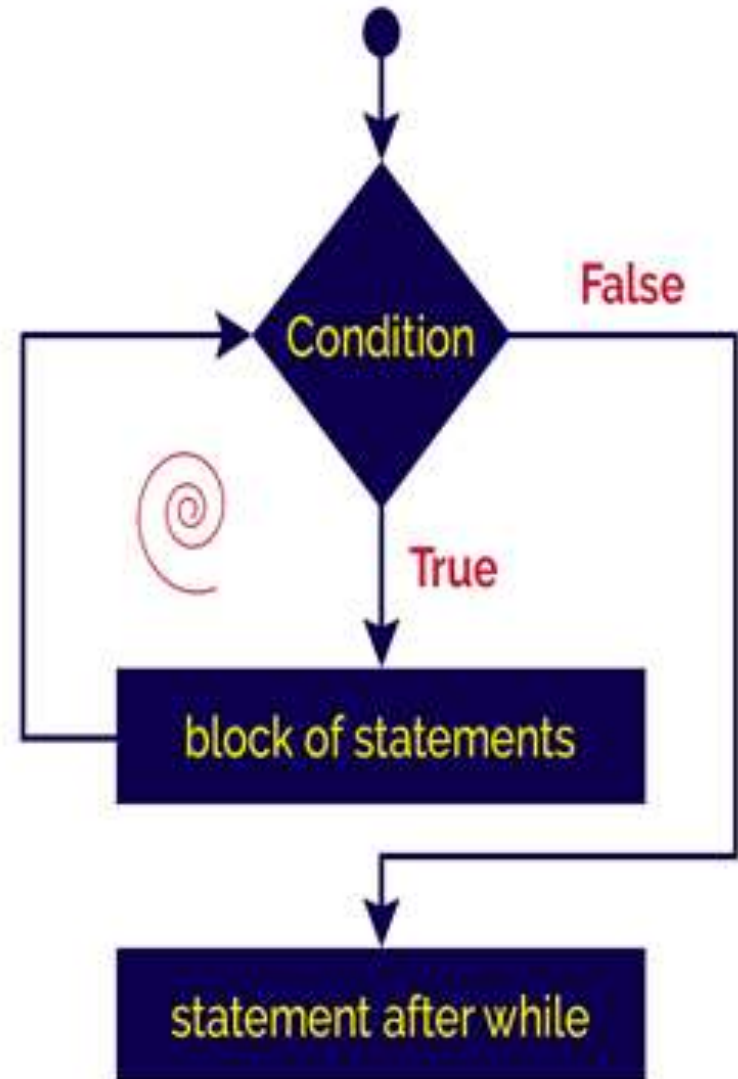
while statement

- The while statement is used to execute a single statement or block of statements repeatedly as long as the given condition is TRUE.
- The while statement is also known as Entry control looping statement. The syntax and execution flow of while statement is as follows.

Syntax

```
while(boolean-expression){  
    block of statements;  
    ...  
}  
statement after while;  
...
```

Flow of execution



WhileTest.java

```
1
2 public class WhileTest {
3
4     public static void main(String[] args) {
5
6         int num = 1;
7
8         while(num <= 10) {
9             System.out.println(num);
10            num++;
11        }
12
13        System.out.println("Statement after while!");
14
15    }
16
17 }
18
```

Console

<terminated> WhileTest [Java Application] C:\Program Files\Java\jre1.8.0_...

```
1
2
3
4
5
6
7
8
9
10
Statement after while!
```

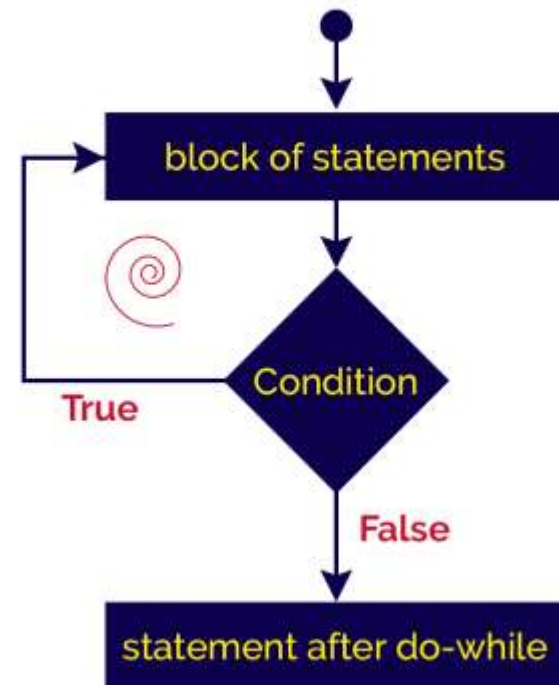
do-while statement

- The do-while statement is used to execute a single statement or block of statements repeatedly as long as given the condition is TRUE.
- The do-while statement is also known as the Exit control looping statement.

Syntax

```
do{  
    block of statements;  
}while(boolean-expression);  
statement after do-while;  
...
```

Flow of execution





DoWhileTest.java

```
1
2 public class DoWhileTest {
3
4     public static void main(String[] args) {
5
6         int num = 1;
7
8         do {
9             System.out.println(num);
10            num++;
11        }while(num <= 10);
12
13        System.out.println("Statement after do-while!");
14
15    }
16
17 }
18
```

Console

<terminated> DoWhileTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe {

```
1
2
3
4
5
6
7
8
9
10
Statement after do-while!
```

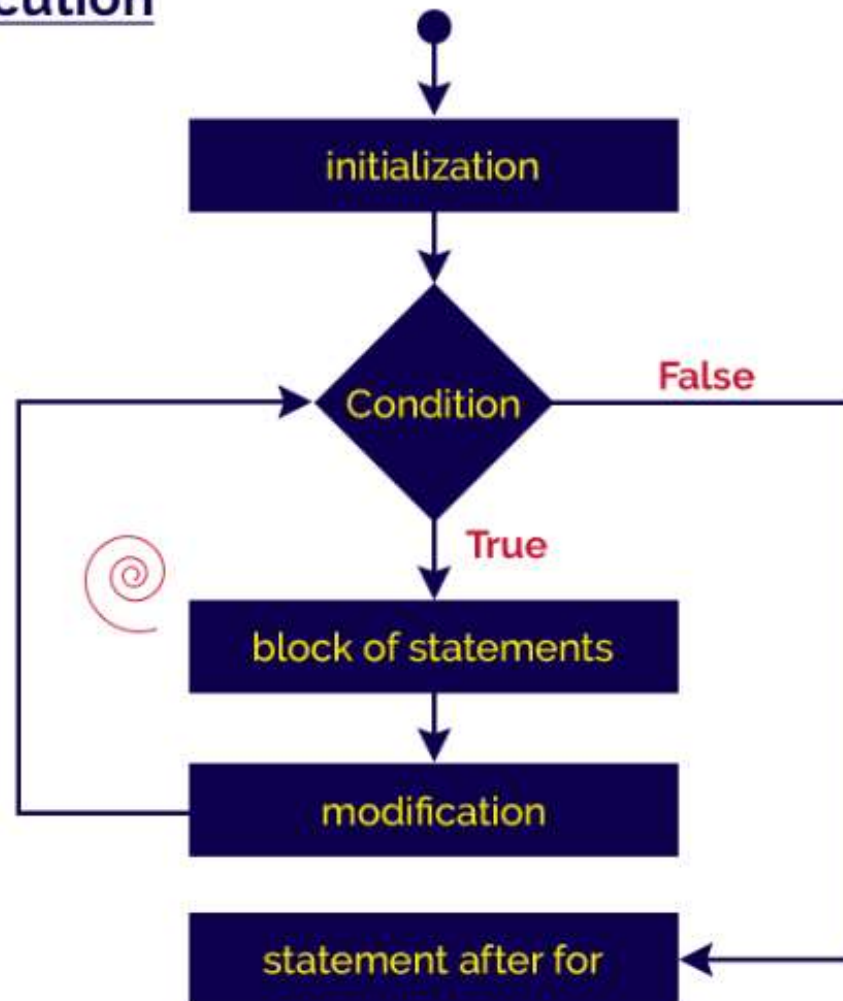
for statement

The for statement is used to execute a single statement or a block of statements repeatedly as long as the given condition is TRUE.

Syntax

```
for(initialization; boolean-expression; modification){  
    block of statements;  
    ...  
}  
statement after for;  
...
```

Flow of execution





EnumExample.java ForTest.java WhileTest.java

```
1
2 public class ForTest {
3
4     public static void main(String[] args) {
5
6         for(int i = 0; i < 10; i++) {
7             System.out.println("i = " + i);
8         }
9
10        System.out.println("Statement after for!");
11    }
12
13 }
14 |
```

Console

<terminated> ForTest [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw

```
i = 0
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
Statement after for!
```

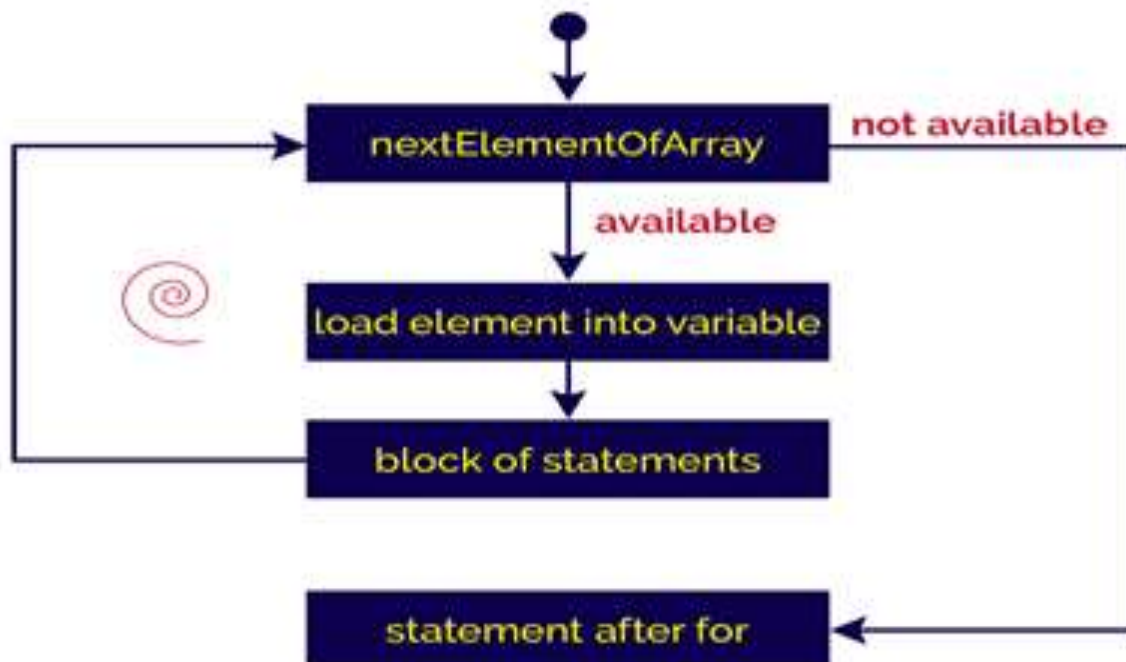
for-each statement

- It provides an approach to traverse through an array or collection in Java.
- The **for-each statement** also known as **enhanced for** statement. The for-each statement executes the block of statements for each element of the given array or collection.

Syntax

```
for( dataType variableName : Array ){  
    block of statements;  
    ...  
}  
statement after for;  
...
```

Flow of execution





EnumExample.java ForEachTest.java WhileTest.java

```
1
2 public class ForEachTest {
3
4     public static void main(String[] args) {
5
6         int[] arrayList = {10, 20, 30, 40, 50};
7
8         for(int i : arrayList) {
9             System.out.println("i = " + i);
10        }
11
12        System.out.println("Statement after for-each!");
13    }
14
15 }
```

Quick Access

<terminated> ForEachTest [Java Application] C:\Program Files\Java\jre1.8

i = 10

i = 20

i = 30

i = 40

i = 50

Statement after for-each!

jump statements

- The java programming language supports jump statements that used to transfer execution control from one line to another line. The java programming language provides the following jump statements.
 - break statement
 - continue statement
 - labelled break and continue statements
 - return statement

break statement

- The break statement in java is used to terminate a switch or looping statement.
- That means the break statement is used to come out of a switch statement and a looping statement like while, do-while, for, and for-each.

Note- Using the break statement outside the switch or loop statement is not allowed.

```
while ( condition)
{
    ...
    break ;
    ...
}

do
{
    ...
    break ;
    ...
} while ( condition) ;
```



The diagram illustrates the execution flow of the provided code. A red arrow originates from the 'break ;' statement within the first 'while' loop and points to the closing brace '}', indicating an immediate exit from the loop. Another red arrow originates from the 'break ;' statement within the 'do-while' loop and points to the closing brace '}', indicating an immediate exit from the loop.

```
for (initilization; condition; modification)
{
    ...
    break ;
    ...
}
```



The diagram illustrates the execution flow of the provided code. A red arrow originates from the 'break ;' statement within the 'for' loop and points to the closing brace '}', indicating an immediate exit from the loop.



JavaBreakStatement.java

```
1
2 public class JavaBreakStatement {
3
4     public static void main(String[] args) {
5
6         int list[] = {10, 20, 30, 40, 50};
7
8         for(int i : list) {
9             if(i == 30)
10                break;
11             System.out.println(i);
12         }
13
14     }
15
16 }
17
```

Console

<terminated> JavaBreakStatement [Java Application] C:\Program Files\Ja

10

20

continue statement

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.
- The continue statement is used to move the execution control to the beginning of the looping statement. When the continue statement is encountered in a looping statement, the execution control skips the rest of the statements in the looping block and directly jumps to the beginning of the loop. The continue statement can be used with looping statements like while, do-while, for, and for-each.
- When we use continue statement with while and do-while statements, the execution control directly jumps to the condition. When we use continue statement with for statement the execution control directly jumps to the modification portion (increment/decrement/any modification) of the for loop

```
while ( condition)
{
    ...
    continue;
    ...
}
do
{
    ...
    continue;
    ...
    ...
} while ( condition);
```



```
for (initilization; condition; modification)
{
    ...
    continue;
    ...
}
```





JavaContinueStatement.java

```
1
2 public class JavaContinueStatement {
3
4     public static void main(String[] args) {
5
6         int list[] = {10, 20, 30, 40, 50};
7
8         for(int i : list) {
9             if(i == 30)
10                continue;
11             System.out.println(i);
12         }
13
14     }
15
16 }
```

Console

<terminated> JavaBreakStatement [Java Application] C:\Program Files\Ja

10

20

40

50

- earning

